REACT · JEST · CYPRESS

Deep dive in practical frontend testing

Unit, integration, and end-to-end tests for a React app

Jest 30 · jest.fn() · Testing Library · Cypress 16
React 19 · Vite · React Router

Examples use a fee total, a sign-in form, and a catalogue.
The same patterns apply to any React storefront on this stack.

1. What the three levels prove

A frontend test is evidence about the page. It shows that a calculation is right, that a component talks to the function it was given, or that a person can finish a journey in the browser. A test that renders a component and checks nothing proves nothing.

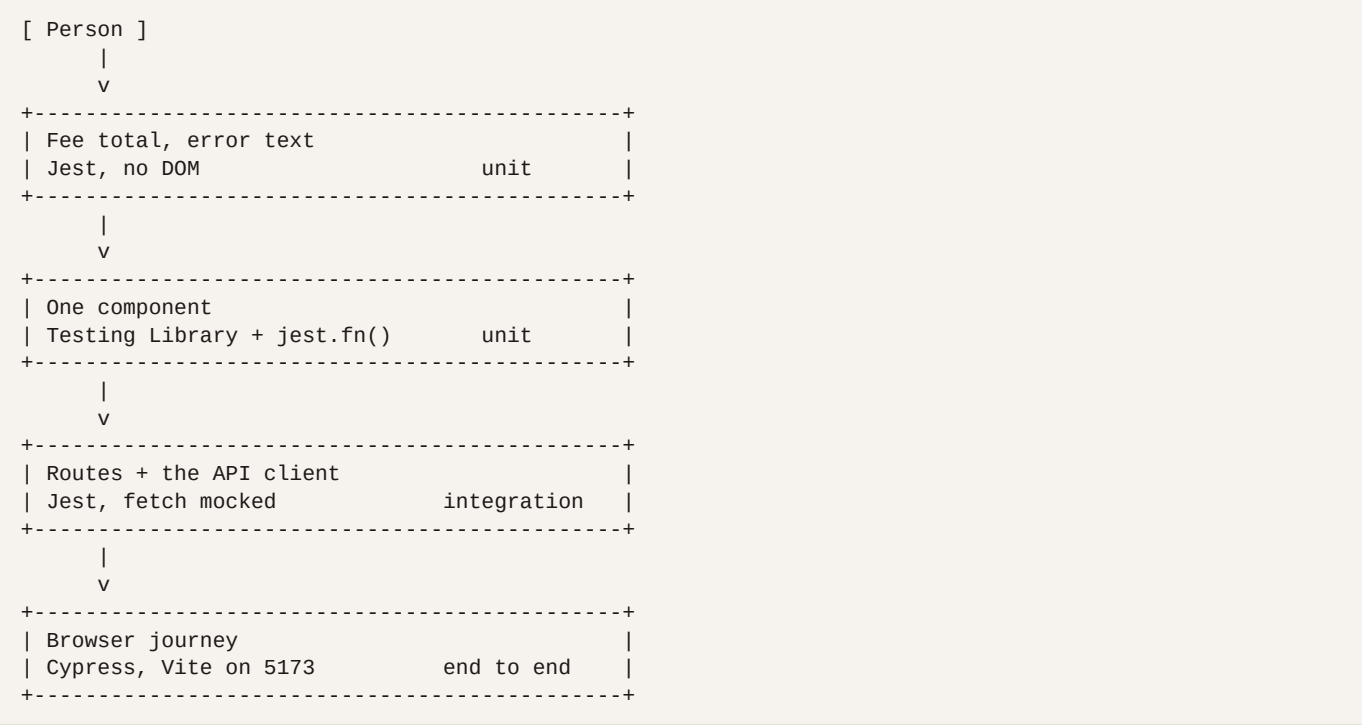
The level is the boundary. Jest and jsdom stay inside one process. Cypress opens a real browser against the Vite dev server. The dev server's proxy to the API is invisible to Jest, which is why a fetch in a Jest test is replaced, and why Cypress is the test that can see the proxy.

	Unit	Integration	End to end
Proves	One function, or one component	Components together, with the router or the API client	One journey in a real browser
Runner	Jest	Jest, still in jsdom	Cypress
Network	None. Pass jest.fn()	fetch is a jest.fn()	The dev server. cy.intercept may script the API
A failure looks like	A wrong total, or a click that never called back	A missing redirect, or a list that ignored the JSON	A button, a label, a proxy, or a screen that disagrees with the API

Put each behaviour at the lowest level that can go red for that bug. A ten-percent fee does not need a browser. A label that is not tied to its input does not need the Java API. Add Cypress only where the browser, the route, or the proxy can be wrong.

2. Where each test sits

A pure function takes values and returns a value. A component receives its collaborators as props. The test passes `jest.fn()` for those props. The page that loads products calls the real client, and the test replaces `fetch`. Cypress is the only row that opens Chrome.



Read the diagram from the bug outward. A wrong total stops at the first row. A button that does not call `onPay` stops at the component row. A catalogue that drops a field from the JSON stops at the integration row. A Pay button that never requests `/api` is the Cypress row.

3. Names, and Arrange - Act - Assert

Every test has three blocks. Arrange prepares inputs and mocks. Act calls the function, clicks, or visits. Assert checks the value, the text on the screen, or the calls recorded by `jest.fn()`.

The name is the sentence a failure prints. Three parts, separated by underscores: `subject_condition_outcome`. Jest uses that string as the function name or as the title of `test()`. Cypress uses it as the title of `it()`. One outcome per test. A title that joins two results with "and" is two tests.

```
test("total_fiveItems_appliesTenPercent", () => {
  // Arrange
  const quantity = 5;
  const unitPrice = 100;

  // Act
  const sum = total(quantity, unitPrice);

  // Assert
  expect(sum).toBe(450);
});
```

4. The four tools

Tool	Job
Jest	Runs the test, provides expect, and loads jsdom so React has a document.
jest.fn()	A stand-in function. It records every call and can return a value you choose.
Testing Library	Renders a component and finds it the way a person does: role, label, text.
Cypress	Drives Chrome against the Vite server. Assertions retry until they pass or time out.

`jest.fn()` is the `fn` in a Jest test. `expect(sum).toBe(450)` checks a value you hold. `expect(onPay).toHaveBeenCalledWith(85000)` checks the conversation with a collaborator. Both sit in the Assert block. `mockResolvedValue` sits in Arrange, because it decides what the next call returns.

Testing Library's `getByRole` and `getByLabelText` fail when the element is missing. `queryByRole` returns null, which is how you assert that a message is absent. `findByRole` waits and returns a promise, which is how you assert text that appears after a fetch. A CSS class is the last resort. A class changes when the layout changes, and the behaviour may still be right.

Jest assertions read once. Cypress commands retry. `cy.get(...).should("be.visible")` keeps looking. A `cy.wait(3000)` either returns too early or wastes the three seconds. Wait on an intercept alias, or on the assertion.

5. Jest on a Vite app

This stack is React 19, built by Vite. Jest does not read `vite.config.js`. Babel compiles JSX for Jest. The proxy that forwards `/api` to port 8080 exists only while `npm run dev` is running, so a Jest test that calls `fetch("/api/...")` must replace `fetch`.

Versions below are the ones this guide is written against. React Testing Library 16 also needs `@testing-library/dom`.

```
npm install --save-dev \
  jest@30.5.2 \
  jest-environment-jsdom@30.5.2 \
  babel-jest@30.5.2 \
  @babel/core@8.0.7 \
  @babel/preset-env@8.0.7 \
  @babel/preset-react@8.0.1 \
  @testing-library/react@16.3.3 \
  @testing-library/dom@10 \
  @testing-library/jest-dom@7.0.1 \
  @testing-library/user-event@14.6.8 \
  cypress@16.2.0 \
  @testing-library/cypress@10.1.3
```

```
{
  "scripts": {
    "dev": "vite",
    "test": "jest",
    "e2e": "cypress run",
    "e2e:open": "cypress open"
  },
  "jest": {
    "testEnvironment": "jsdom",
    "setupFilesAfterEnv": [
      "<rootDir>/src/setupTests.js"
    ],
    "transform": {
      "^.+\\.jsx?$": "babel-jest"
    }
  }
}
```

```
{
  "presets": [
    ["@babel/preset-env",
      { "targets": { "node": "current" } }],
    ["@babel/preset-react",
      { "runtime": "automatic" }]
  ]
}
```

Save that object as `babel.config.json`. `src/setupTests.js` is one import: `import "@testing-library/jest-dom"`. That import adds `toBeInTheDocument`. The React preset's automatic runtime matches the JSX this app already uses, so test files do not import React.

6. Unit tests of a function

The fee rule is the same idea as on the server. Five items take ten percent off. Four items pay full price. A negative quantity is refused. Integer division stays integer: `Math.floor` so 1 is not invented by a float.

```
export function total(quantity, unitPrice) {
  if (quantity < 0 || unitPrice < 0) {
    throw new Error(
      "quantity and unitPrice must be non-negative");
  }
  const subtotal = quantity * unitPrice;
  return quantity >= 5
    ? Math.floor((subtotal * 90) / 100)
    : subtotal;
}
```

```
test("total_fourItems_chargesFullPrice", () => {
  expect(total(4, 100)).toBe(400);
});

test("total_fiveItems_appliesTenPercent", () => {
  expect(total(5, 100)).toBe(450);
});

test("total_negativeQuantity_throws", () => {
  expect(() => total(-1, 100)).toThrow(
    /non-negative/);
});
```

An API error often arrives as `{ detail }` or `{ title }`, and sometimes as an empty body. The helper that turns that into a sentence is a pure function. Test the three bodies. There is nothing to mock.

```
export function problemMessage(body, fallback) {
  if (!body) return fallback;
  return body.detail || body.title || fallback;
}

test("problemMessage_detail_returnsDetail", () => {
  expect(problemMessage(
    { detail: "Loan not found" }, "failed"))
    .toBe("Loan not found");
});

test("problemMessage_empty_returnsFallback", () => {
  expect(problemMessage(null, "failed")).toBe("failed");
});
```

7. jest.fn(), the stand-in

A component that pays does not contain the payment rule. It receives `onPay`. The test passes `jest.fn()` so it can ask, afterwards, whether the click happened and what was passed. The real `onPay` is a different test, higher up, where the API client runs.

```
const onPay = jest.fn();
onPay.mockReturnValue("ok");

onPay(85000);

expect(onPay).toHaveBeenCalledTimes(1);
expect(onPay).toHaveBeenCalledWith(85000);
expect(onPay).toHaveReturnedWith("ok");
```

Call	When
jest.fn()	Record calls. The default return value is undefined.
mockReturnValue(x)	The next call returns x. Arrange, for a sync collaborator.
mockResolvedValue(x)	The next call returns a promise of x. Arrange, for async.
mockRejectedValue(err)	The next call rejects. Arrange, for the failure path.
toHaveBeenCalledWith(...)	Assert. Those arguments were passed.
toHaveBeenCalledTimes(n)	Assert. It ran n times. Zero is a real assertion.
mockClear()	Forget the recorded calls. Keep the implementation.
mockReset()	Forget the calls and the mockReturnValue.

Put `jest.clearAllMocks()` in `beforeEach` when several tests share one `jest.fn()`. A later test then starts from zero calls. Clearing is not a substitute for a new `jest.fn()` inside the test that owns it.

Mock the collaborator. Leave the function under test real. A `jest.fn()` assigned over `total` proves the mock, and the ten-percent rule never runs.

8. One component

The button's job is to call `onPay` with the amount when a person clicks it. `userEvent` clicks the way a person does. Find the button by its accessible name.

```
export function PayButton({ amount, onPay }) {
  return (
    <button type="button"
            onClick={() => onPay(amount)}>
      Pay {amount}
    </button>
  );
}
```

```
import { render, screen } from "@testing-library/react";
import userEvent from "@testing-library/user-event";

test("pay_click_callsOnPayWithTheAmount", async () => {
  // Arrange
  const user = userEvent.setup();
  const onPay = jest.fn();
  render(<PayButton amount={85000} onPay={onPay} />);

  // Act
  await user.click(
    screen.getByRole("button", { name: "Pay 85000" }));

  // Assert
  expect(onPay).toHaveBeenCalledTimes(1);
  expect(onPay).toHaveBeenCalledWith(85000);
});
```

The sign-in form owns the email, the password, and the error text. `onSignIn` is a `jest.fn()`. A resolved mock is the success path. A rejected mock is the message path. The label uses `htmlFor` matching the input's `id`. A label that merely sits beside an input is invisible to `getByLabelText`. Fix the label. Do not switch the test to a CSS class.

```
export function SignInForm({ onSignIn }) {
  const [email, setEmail] = useState("");
  const [password, setPassword] = useState("");
  const [error, setError] = useState("");

  async function onSubmit(event) {
    event.preventDefault();
    if (!email.trim()) {
      setError("Email is required");
      return;
    }
    setError("");
    try {
      await onSignIn(email, password);
    } catch (err) {
      setError(err.message);
    }
  }

  return (
    <form onSubmit={onSubmit}>
      <label htmlFor="email">Email</label>
      <input id="email" value={email}
        onChange={(e) => setEmail(e.target.value)} />
      <label htmlFor="password">Password</label>
      <input id="password" type="password"
        value={password}
        onChange={(e) => setPassword(e.target.value)} />
      <button type="submit">Sign in</button>
      {error ? <p role="alert">{error}</p> : null}
    </form>
  );
}
```

```
test("signIn_blankEmail_showsTheMessage", async () => {
  // Arrange
  const user = userEvent.setup();
  const onSignIn = jest.fn();
  render(<SignInForm onSignIn={onSignIn} />);

  // Act
  await user.click(
    screen.getByRole("button", { name: "Sign in" }));

  // Assert
  expect(screen.getByRole("alert"))
    .toHaveTextContent("Email is required");
  expect(onSignIn).not.toHaveBeenCalled();
});

test("signIn_rejected_showsTheServerMessage", async () => {
  const user = userEvent.setup();
  const onSignIn = jest.fn().mockRejectedValue(
    new Error("Email or password is incorrect"));
  render(<SignInForm onSignIn={onSignIn} />);

  await user.type(
    screen.getByLabelText("Email"), "ada@shop.test");
  await user.type(
    screen.getByLabelText("Password"), "nope");
  await user.click(
    screen.getByRole("button", { name: "Sign in" }));

  expect(await screen.findByRole("alert"))
    .toHaveTextContent("Email or password is incorrect");
  expect(onSignIn).toHaveBeenCalledWith(
    "ada@shop.test", "nope");
});
```

`findByRole` is the assertion that waits for the alert after the promise rejects. `getByRole` would read the document once, before React paints the message. `queryByRole("alert")` is the check you use when the success path must show no alert: `expect(screen.queryByRole("alert")).not.toBeInTheDocument()`.

9. Integration: router and fetch

An integration test crosses a boundary inside Jest. Two boundaries matter on this app. The router decides which page a URL shows. The API client decides what `fetch` sends and how a failed response becomes an error. jsdom has no Vite proxy, so `fetch` is `jest.fn()`.

9.1 A guard sends an anonymous visitor to sign in

The guard reads a token. No token means the sign-in route. `MemoryRouter` supplies the URL without a browser. This is the integration of the guard and the route table. It does not prove the address bar of Chrome, and it does not prove the proxy.

```
function RequireAuth({ token, children }) {
  if (!token) return <Navigate to="/login" replace />;
  return children;
}

test("orders_anonymous_showsSignIn", () => {
  // Arrange + Act
  render(
    <MemoryRouter initialEntries={["/orders"]}>
      <Routes>
        <Route path="/login"
          element={<h1>Sign in</h1>} />
        <Route path="/orders" element={
          <RequireAuth token={null}>
            <h1>Orders</h1>
          </RequireAuth>
        } />
      </Routes>
    </MemoryRouter>
  );

  // Assert
  expect(screen.getByRole("heading",
    { name: "Sign in" })).toBeInTheDocument();
  expect(screen.queryByRole("heading",
    { name: "Orders" })).
    not.toBeInTheDocument();
});
```

The storefront reads the token from its store instead of a prop. Pass the store's empty token the same way: when nobody is signed in, the value the guard sees is null. A signed-in test passes a token and expects the Orders heading.

9.2 The client, with fetch replaced

The client reads the body with `text()`, then parses JSON. A mock that only provides `json()` does not match this client. The fake response below has the methods the client calls. `headers.get` returns null unless the test is about a header such as a guest cart token.

```
export async function loadCatalogue() {
  const res = await fetch("/api/v1/products");
  const text = await res.text();
  const data = text ? JSON.parse(text) : null;
  if (!res.ok) {
    const err = new Error(
      problemMessage(data, "Request failed"));
    err.status = res.status;
    throw err;
  }
  return data;
}

function jsonResponse(status, body) {
  return {
    ok: status >= 200 && status < 300,
    status,
    headers: { get: () => null },
    text: async () => JSON.stringify(body),
  };
}
```

```
beforeEach(() => {
  global.fetch = jest.fn();
});

test("loadCatalogue_ok_returnsTheProducts", async () => {
  // Arrange
  fetch.mockResolvedValueOnce(jsonResponse(200, [
    { sku: "USB-C-01", name: "USB-C cable" },
  ]));

  // Act
  const products = await loadCatalogue();

  // Assert
  expect(products).toEqual([
    { sku: "USB-C-01", name: "USB-C cable" },
  ]);
  expect(fetch).toHaveBeenCalledTimes(1);
  expect(fetch).toHaveBeenCalledWith(
    "/api/v1/products");
});

test("loadCatalogue_notFound_throwsTheDetail", async () => {
  fetch.mockResolvedValueOnce(
    jsonResponse(404, { detail: "Not found" }));

  await expect(loadCatalogue())
    .rejects.toThrow("Not found");
});
```

The catalogue component calls that real function. The test still owns `fetch`. `findByRole` waits until the list item exists. This is the integration of the component and the client. It is not a test of the Java controller, because the JSON was written by the mock.

```
export function Catalogue() {
  const [items, setItems] = useState([]);
  const [error, setError] = useState("");

  useEffect(() => {
    loadCatalogue()
      .then(setItems)
      .catch((err) => setError(err.message));
  }, []);

  if (error) return <p role="alert">{error}</p>;
  return (
    <ul>
      {items.map((item) => (
        <li key={item.sku}>{item.name}</li>
      ))}
    </ul>
  );
}

test("catalogue_oneProduct_listsTheName", async () => {
  fetch.mockResolvedValueOnce(jsonResponse(200, [
    { sku: "USB-C-01", name: "USB-C cable" },
  ]));
  render(<Catalogue />);

  expect(await screen.findByText("USB-C cable"))
    .toBeInTheDocument();
});
```

10. Cypress, the browser

Cypress opens Chrome, visits the Vite server on port 5173, and drives the page. The app is the one `npm run dev` serves, including React Router and the proxy in `vite.config.js`. Jest never started that server.

Cypress Component Testing can mount a component too. This guide keeps components in Jest, so a button is not tested twice. Cypress is the journey.

10.1 Install the commands and point at Vite

```
import { defineConfig } from "cypress";

export default defineConfig({
  e2e: {
    baseUrl: "http://127.0.0.1:5173",
    supportFile: "cypress/support/e2e.js",
  },
});
```

```
import "@testing-library/cypress/add-commands";

// cypress/support/e2e.js
// cy.findByRole and cy.findByLabelText
// are then available in every spec.
```

`cy.findByRole` comes from `@testing-library/cypress`. It retries, and it uses the same role and label queries as the Jest tests. The dev server has to be up before `cy.visit`. `start-server-and-test` starts Vite,

waits for the port, runs Cypress, and stops Vite.

```
npm install --save-dev start-server-and-test

npx start-server-and-test dev http://127.0.0.1:5173 \
  "cypress run"
```

10.2 A journey with the API scripted

`cy.intercept` answers the browser's request with JSON you wrote. The button, the route, and the render are real. The Java controller is not. Use this when the bug is on the page. Give the intercept an alias and `cy.wait("@alias")` so the assertion runs after the response, not after a guessed delay.

```
describe("catalogue", () => {
  it("home_oneProduct_showsTheName", () => {
    cy.intercept("GET", "/api/v1/products", {
      statusCode: 200,
      body: [
        { sku: "USB-C-01", name: "USB-C cable" },
      ],
    }).as("products");

    cy.visit("/");
    cy.wait("@products");

    cy.findByText("USB-C cable")
      .should("be.visible");
  });
});
```

```
it("login_validCustomer_reachesHome", () => {
  cy.intercept("POST", "/api/v1/auth/login", {
    statusCode: 200,
    body: { token: "test-token" },
  }).as("login");

  cy.visit("/login");
  cy.findByLabelText("Email").clear()
    .type("ada@shop.test");
  cy.findByLabelText("Password").clear()
    .type("Secret1a");
  cy.findByRole("button", { name: "Sign in" })
    .click();

  cy.wait("@login").its("request.body")
    .should("deep.include", {
      email: "ada@shop.test",
    });
  cy.url().should("eq",
    "http://127.0.0.1:5173/");
});
```

If `findByLabelText("Email")` times out, the Email label is not associated with the input. Wrap the input in the label, or set `htmlFor` to the input's id. The password field on a form that already uses `htmlFor` is the pattern to copy.

10.3 The same guard, in the address bar

The Jest test of `RequireAuth` already showed the sign-in heading inside jsdom. Cypress repeats the anonymous visit only because the address bar and the real router are now in play. It still does not need the Java API: the guard runs in the browser before any order request.

```
it("orders_anonymous_landsOnLogin", () => {
  cy.visit("/orders");

  cy.findByRole("heading", { name: "Sign in" })
    .should("be.visible");
  cy.url().should("include", "/login");
});
```

10.4 A journey that hits the real API

Leave the intercept out when the bug might be the proxy, the token header, or the JSON the server actually returns. Vite must proxy `/api` to the Spring Boot port, and that API must be the test stack, with a test user you created for the suite. A developer's database, and a server left on 8080 from yesterday, make the journey depend on whatever was left there.

Keep that set small. The fee total already has a Jest test. The login JSON already has a server test. This journey checks that a person can sign in and that the screen shows a status the server stored. `cy.intercept` is the wrong tool for that last fact, because the intercept is the status.

11. Which tool answers which bug

Bug	Test that goes red
Five items billed at full price	total_fiveItems_appliesTenPercent in Jest.
Pay click does not pass the amount	The PayButton test and jest.fn().
Blank email still calls the API	signIn_blankEmail_showsTheMessage. not.toHaveBeenCalled().
Rejected login shows a blank page	The alert test. findByRole waits for the message.
Anonymous /orders renders orders	The MemoryRouter guard test.
The client ignores detail on a 404	loadCatalogue_notFound_throwsTheDetail.
The list never renders the product name	The Catalogue test, fetch mocked.
The Email label is not tied to the input	Cypress findByLabelText times out.
The button posts to /api/product	Cypress, with or without an intercept. The URL of the request is wrong.
The proxy still points at port 8080	Cypress with the intercept omitted, against the test API.

12. What a Jest test cannot prove

- **The Vite proxy.** Jest never starts Vite. A green fetch mock says the client sent the path you asserted.
- **The Java controller.** The JSON in `mockResolvedValue` and in `cy.intercept` was written by the test.
- **Layout.** jsdom does not place pixels. A Cypress screenshot is a separate choice. This guide asserts text, role, and URL.

- **A real card charge.** The payment result in these tests is a value the test supplied.
- **Both outcomes in one title.** A test named `signIn_blank_showsErrorAndCallsApi` is two tests.

13. Running one level

```
cd frontend
npm test
npm run e2e:open
npx start-server-and-test dev http://127.0.0.1:5173 \
  "cypress run"
```

`npm test` is Jest: unit and integration, no Chrome, no Docker. Cypress needs the dev server. The full-stack specs also need the API the proxy targets. A pipeline can split them the same way the server suite splits groups.

```
unit:
  script:
    - cd frontend && npm ci && npm test

e2e:
  script:
    - cd frontend && npm ci
    - npx start-server-and-test dev
        http://127.0.0.1:5173 "npx cypress run"
```

The unit job has no browser. The Cypress job installs the Cypress binary on first run, which is large, so keep it on the end-to-end job. Specs that omit `cy.intercept` belong on a job that also starts the API.

14. Before you call a test finished

- The name is subject_condition_outcome, in Jest and in Cypress.
- Arrange, act, and assert are in that order.
- A pure function has no jest.fn(). A collaborator is a jest.fn().
- The query is getByRole or getByLabelText. queryBy asserts absence. findBy waits.
- A label has htmlFor, or it wraps the input.
- fetch mocks include text() and headers.get when the client uses them.
- Cypress waits on an alias or on should(). It does not wait on a fixed sleep.
- The fee stays in Jest. The button stays in Cypress. The proxy is Cypress without an intercept.

15. Questions

One option is correct. Where a question states the fee rule, five items take ten percent off and four items pay full price.

1. A unit test of total(5, 100) starts which of these?

A. Chrome
B. The Vite dev server
C. Jest, which calls the function
D. The Spring API

2. jest.fn() is used so the test can

A. Replace the function under test and skip it
B. Record how a collaborator was called, and choose what it returns
C. Compile JSX
D. Open port 5173

3. expect(onPay).toHaveBeenCalledWith(85000) checks

A. The text of the button
B. The arguments passed to the mock
C. The HTTP status
D. The CSS class

4. mockResolvedValue belongs in

A. Assert
B. Arrange
C. The Cypress config
D. babel.config.json

5. A blank email must not call onSignIn. The assertion is

A. expect(onSignIn).toHaveBeenCalled()
B. expect(onSignIn).not.toHaveBeenCalled()
C. cy.wait(3000)
D. expect(true).toBe(true)

6. getByRole throws when the element is missing. To assert that an alert is absent, use

A. getByRole
B. queryByRole
C. findByRole
D. cy.wait(0)

7. findByRole is the query that

A. Reads the document once
B. Waits until the element appears
C. Clicks the element
D. Mocks fetch

8. A label with no htmlFor and no wrapped input makes getByLabelText

A. Match the nearest input
B. Fail to find the field
C. Match by CSS class
D. Skip the test

9. MemoryRouter plus RequireAuth is which level?

A. End to end, because a URL changed
B. Integration of the guard and the routes, inside jsdom
C. A Cypress spec
D. A pure unit test of total()

10. A Jest test of loadCatalogue proves the Java controller when

A. fetch is mockResolvedValue with JSON the test wrote
B. It never does. The mock is the response

C. The file is named .cy.js
D. jsdom starts Spring Boot

11. The fake fetch response for this client must provide

A. only json()
B. text() and headers.get, because the client calls those
C. a Cypress alias
D. localStorage

12. Vite's proxy to port 8080 is visible to

A. Every Jest test
B. Cypress, while npm run dev is serving the app
C. babel-jest
D. jest.fn()

13. cy.intercept("GET", "/api/v1/products", ...) proves

A. The Spring controller's JSON
B. That the page renders the body the intercept returned
C. The fee rule
D. Pixel layout

14. cy.wait(3000) before an assertion

A. Is how Cypress retries
B. Waits a fixed time. Prefer cy.wait("@alias") or .should()
C. Replaces findByRole in Jest
D. Starts the API

15. orders_anonymous_landsOnLogin in Cypress adds, over the Jest guard test,

A. A proof of the ten-percent fee
B. The real address bar and the running router
C. A mock of onPay
D. jsdom

16. A full-stack Cypress spec omits cy.intercept when

A. The proxy or the server's real JSON might be the bug
B. The fee might be wrong
C. The button's onClick might be missing
D. Jest is not installed

17. Which title matches subject_condition_outcome?

A. testPay
B. pay_click_callsOnPayWithTheAmount
C. it works
D. PayButton

18. signIn_blank_showsErrorAndSubmits is

A. One test with two outcomes
B. Two tests
C. A Cypress-only name
D. A snapshot

19. npm test on this setup runs

A. Cypress against port 5173
B. Jest in jsdom
C. The Spring suite
D. Vite's production build

20. The ten-percent fee belongs in

A. A Cypress screenshot
B. total_fiveItems_appliesTenPercent
C. cy.intercept
D. a CSS snapshot

16. Answers

1. C. total is a pure function. Jest calls it. The browser and the API stay down.

2. B. The collaborator is the mock. The function or component under test stays real.

3. B. toHaveBeenCalledWith is the assertion about the conversation. toBe checks a value you hold.

4. B. It decides what the next async call returns, before Act.

5. B. Zero calls is a real outcome. The alert text is the other assertion, in its own check.

6. B. queryByRole returns null. not.toBeInTheDocument() states the absence.

7. B. The alert after a rejected promise is painted later. findByRole waits. getByRole does not.

8. B. The accessible name is missing. Associate the label. Leave the query as getByLabelText.

9. B. There is no Chrome and no proxy. Two React pieces collaborate.

10. B. The controller is proved by a server test, or by Cypress with the intercept left out.

11. B. A mock of json() does not satisfy a client that reads text().

12. B. Jest does not start Vite, so it never loads vite.config.js.

13. B. The body was written by the test. The page's use of that body is what can go red.

14. B. Cypress retries .should(). A fixed wait returns too early or wastes the delay.

15. B. The guard's branch is already tested in Jest. Cypress checks the URL the browser shows.

16. A. An intercept would hide the proxy and the controller. Keep these specs few, and point them at a test API.

17. B. Subject, condition, outcome. The same three parts title a Cypress it().

18. B. A name that joins two results is two tests.

19. B. The test script is jest. Cypress is npm run e2e, with the dev server up.

20. B. The rule is a pure function. The browser is the wrong boundary.